CS673 Final Product Report

Project Name: GradAid

Group Name: Admissions Alchemists

Github URL: https://github.com/yuezhang0594/GradAid

Product URL: https://gradaid.online/

Original Product Requirements (Functional Features)

Must Have – minimal for release, required for a useful product

Feature	User Story	Completed by team member(s) or reason not completed
Interactive form to collect user information and store in a database.	As an aspiring graduate student, I need to provide information about myself through an interactive form.	Yue Zhang wrote the majority of code for this feature, Joseph Rissman supported with code review and backend tests.
User authentication.	As a multi-device user, I need to log in so that my activity and history are saved and synchronized across all devices.	Joseph Rissman set up user authentication through Clerk as a microservice and integrated it with the database and the UI.
Customized SOP generation, specific to university requirements.	As an applicant applying to many graduate programs, I need automated document creation based on my university choice so that I can focus more on my application strategy.	Yue Zhang wrote the code for generating documents with the OpenAI API, using profile and program-specific information from the database. Nitin Krishna Bojji created a custom prompt to provide more personalized responses.
Customized SOP generation, specific to user information..	As a mid-career professional seeking a graduate degree abroad, I need to have my diverse background and skillset reflected in my SOP.	See above.

Cross-platform support.	As a global applicant with limited internet access, I need to interact with GradAid in a web browser on my phone.	Yue Zhang set up the project on Netlify to host the product on the internet. Yue Zhang and Joseph Rissman both contributed to ensuring the UI and UX is high-quality for both desktop and mobile users.
User authentication and data security (RBAC, OAuth, etc.)	As a privacy-conscious user, I need data encryption and security measures so that my personal information is protected from unauthorized access and potential leaks.	Joseph Rissman set up automated routing to ensure users cannot access the application features unless they are signed in through Clerk. Joseph Rissman and Yue Zhang designed all code to be focused on security, with multiple checks to ensure users can only access their own data. Joseph Rissman implemented a feature to allow users to delete their account and all associated data for extra security.

Want to Have – improves on a required feature or additional functionality

Feature	User Story	Completed by team member(s) or reason not completed
Document management system.	As an applicant needing to update my application materials, I want to quickly find my relevant documents.	Yue Zhang implemented a document page where users can quickly see all of their documents sorted by application. Yue Zhang also implemented routing to allow users to access their documents for an application directly from the application detail page or even straight from the main dashboard.
Document formatting tools.	As an applicant preparing documents for various universities, I want to specify the format of downloaded documents.	This feature was not completed. This was planned for implementation after MVP launch, but MVP launch was pushed back due to delays integrating AI document generation. At present, users can still copy and paste the document contents into a word processor of

		their choice, which is a typical user experience when interacting with AI chatbots.
Application tracking system.	As an applicant managing multiple university applications, I want a system for tracking application deadlines so that I can ensure timely submissions and avoid missing critical dates	Yue Zhang implemented the code for tracking application deadlines and displaying them to the user. Joseph Rissman implemented code to allow users to set custom deadlines for greater control of the application process.
Application tracking system.	As someone that struggles with organization and planning, I want a feature for tracking the progress of my applications so that I can stay informed about their status and manage any required follow-up actions.	Yue Zhang wrote all of the code for this feature, however in the current state it is a bit primitive and not bug-free. Work on this feature was delayed due to Yue needing to work on the AI document generation after it became clear Nitin was not contributing to the project.
Program search.	As a prospective graduate student exploring various academic options, I want to interactively search through university programs and save them to a personalized list.	Joseph Rissman wrote all of the code for graduate program search and save features. This should have been a must-have requirement as it is now a major part of the product that begins the user flow for creating applications and related documents. Nitin Krishna Bojji helped by web scraping data on universities and graduate programs that populated the database. Joseph Rissman contributed to web scraping by filling in missing data and cleaning incorrect data.

Nice to Have – things to do if there is time

Feature	User Story	Completed by team member(s) or reason not completed
Speech-to-text and text-to-speech.	As an applicant with accessibility needs, I want GradAid to have	This feature was not completed. To be honest, I don't understand what this feature was supposed

	speech recognition and speech generation capabilities.	to do. Nitin really wanted this feature, but he did not work on it so it did not happen.
UI/UX design and testing.	As a user with limited computer literacy, I want the GradAid website to be inviting with a modern design.	Yue Zhang and Joseph Rissman both worked on the website design to ensure good UI and UX. The decision was made after the first project update to restart the website with the Shadcn/ui component library to provide the website with a consistent and clean look to it. Many users left positive feedback about the design saying it is “sleek” or “slick” and “easy to navigate” so I believe we accomplished this one.
AI-power document updates.	As someone who has gained a new qualification, I want to be able to update my SOPs and LORs so that my new accomplishment is included.	Yue Zhang wrote all of the code to make the documents editable. Additionally, users can update their profile information if they have new accomplishments or forgot to include something, then regenerate their documents.
University shortlisting.	As a prospective graduate student, I want to receive a shortlist of universities based on my experience and accomplishments.	This feature was not completed. Nitin did write a python function to perform this task, but he delivered it towards the end of the semester. At that point we were still trying to complete the MVP by implementing AI document generation because Nitin had not delivered anything else up to that point. Therefore, the rest of the team did not have the bandwidth to implement a new page for this feature or rewrite the python code so it could actually be used with our database and frontend.

Additional Product Features (Features not anticipated, but completed)

Feature	Reason Implemented	Completed by team member(s)
Landing page with hero section.	Since authentication was required for access to any website features, it became necessary to create a landing page that describes what GradAid is about.	Yue Zhang wrote the majority of code for this feature. Joseph Rissman implemented the sign-in and sign-up flows with Clerk.
Application dashboard.	With the growing complexity of GradAid being split across many pages, we needed a centralized dashboard to display the most important information to the user.	Yue Zhang wrote all of the code for this feature. The dashboard and the associated breakout pages (timeline, credit summary, applications, and activity) are all his work.
User onboarding.	It became clear that to generate personalized documents, we needed a minimum amount of information about the user. Onboarding forces the user to minimally complete their profile before they can access any other core features.	Yue Zhang wrote the code for user onboarding, walking them through the process of creating their profile. Joseph Rissman wrote the code that forces the user to complete onboarding before accessing critical features.
AI usage management.	Because we needed to use the OpenAI API to generate the document content, we needed to limit the amount of documents users can generate to prevent abuse.	Joseph Rissman wrote the backend code that tracks AI usage with credits, and enforces the limit when it is exceeded. Yue Zhang wrote the frontend code that displays the AI usage history and remaining credits to the user.
Logging user activity.	It might be difficult for users to remember what they were doing last if they come back after a while or why they had used up all their AI credits. This feature helps users keep track of their progress.	Joseph Rissman and Yue Zhang wrote the backend code that tracks different user activities. Yue Zhang wrote the frontend code that displays the activity to the user. Admittedly, this feature still has bugs that need to be fixed before it is fully usable.

Frequently asked questions page.	With the growing complexity of the website, it seemed reasonable to include an FAQ page to help users that get lost or have trouble with certain aspects.	Joseph Rissman wrote all of the code for this feature.
Contact form.	If users have any issues with the website, they can directly contact the developers through email to quickly get a response.	Joseph Rissman wrote all of the code for this feature.
Feedback form.	We wanted to collect user feedback to improve the website. Unfortunately, due to a certain team member not contributing to the project, our MVP launch was pushed back too far to be able to actually implement any of the suggested improvements before the end of the semester.	Joseph Rissman wrote all of the code for this feature.
Terms of service & privacy policy.	Although these are not legally binding documents that have not been looked at by a lawyer, it felt like including these legitimized the website.	Joseph Rissman wrote all of the code for this feature.

Team Process (New)

The Admissions Alchemists team began the project at the start of the semester by roughly dividing the responsibilities across the three team members. Yue was to focus on frontend web development, Joseph was to be responsible for backend development and database management, and Nitin would handle the AI-related aspects of the project.

During the first third of the semester (up until the first presentation), each team member developed the product according to their responsibilities. Joseph sourced a database provider and set up some SQL functions to store user data and program information. Yue built out a custom landing page and dashboard. Nitin claims to have been fine-tuning a custom AI model (there is no proof this ever happened).

Following the first mid-semester presentation, it became clear that working independently wasn't allowing development to proceed at the rate it needed to. Together, Joseph and Yue decided to scrap the work that had already been done and start over. This decision was easy because both had learned so much since the beginning of the semester, that it only took about a week to get back to the point they were at before the presentation. By starting over, they both worked

together to select exactly the framework and tools they wanted to use for the project. Joseph decided to move over to Convex due to the simplicity of the database provider and its ability to seamlessly integrate with React. Another advantage of starting over is that now the entire project can be written in TypeScript, which also speeds up the development by forcing frontend and backend code to be written in the same language, making troubleshooting easier. Yue agreed to build the frontend code again in React with TypeScript, using the Shadcn component library to speed up development by implementing reusable components. At this point, Nitin claims to still be working on fine-tuning the LLM.

After Joseph and Yue had agreed to restart the project and work more closely together, the responsibilities became more blurred. Joseph was still in charge of the backend and managing the Convex server and Yue was still in charge of the frontend and managing the Netlify deployment. However now both Joseph and Yue took on roles of full-stack developers, taking responsibility for a feature and generating both frontend and backend code for it. Joseph first took on the feature of program search and save while Yue took on the feature of user profile and onboarding. These features took approximately two sprints to complete as Spring Break fell in the middle of the sprints. Both Joseph and Yue worked concurrently on the website layout and design, with both contributing to the code for the sidebar and header components. Joseph also took on the responsibility of managing user authentication with Clerk. Nitin was still “working” on fine-tuning the custom LLM.

Once the profile and search features were completed, the next features that were implemented were creating a new application and editing documents. Joseph worked on creating new applications, once again writing the frontend and backend code to make it work. Yue worked on the document editor, writing both frontend and backend code to get it working. This work took another sprint, with both Joseph and Yue doing code review for the features the other built in the previous sprints. Nitin claims to have finished fine-tuning the LLM, but couldn’t figure out how to run it anywhere except locally on his computer. He was given the next sprint to try to make an API for his LLM available.

After the create application and edit document features were implemented, they were tied together to finish the user flow. Joseph worked on the frontend and backend code to create new documents while Yue finished writing the frontend and backend code for the dashboard, tying everything together. Once again, Joseph and Yue reviewed the code for the components the other completed. This sprint also saw Joseph and Yue polish the UI/UX by ensuring each feature built by each team member used a new reusable page-wrapper component to keep a consistent look across the website. Nitin was unable to find a way to host his custom LLM (possibly because it didn’t exist), so he was tasked with developing code to use an existing LLM API, such as OpenAI or Llama.

In the penultimate sprint, it became apparent that Nitin was incapable of contributing useful code to the project so Yue took on the responsibility of integrating an LLM API into the codebase. Yue wrote the backend code to use the OpenAI API to generate custom application documents, then integrated it with the frontend code he had already written. Yue took the prompt that Nitin had written previously, which is a critical piece that makes the AI generated

content specific to the user and program. During this time, Joseph focused on creating the support pages, including the FAQ page, contact page, and feedback form. He wrote all of the frontend and backend code for all of these features. Joseph also refactored the Convex server functions to make them more maintainable and follow Convex best practices. Joseph also implemented the AI credit management system. Both Yue and Joseph went back through all of the components they had generated so far and implemented input validation and sanitization. Since Yue completed the AI generation feature, Nitin was tasked with web scraping program and university data to populate the database.

The final sprint saw Joseph and Yue writing tests for all of the code they had generated so far. Joseph took the web scraped data from Nitin, cleaned it, filled in missing data, then populated the database. Joseph and Yue worked together to push everything to production servers and launch the website, providing a week to gather user feedback. There were several small bug fixes and UI updates in the final sprint, with Joseph and Yue reviewing the code the other had written. Nitin wrote a single test for the AI document generation, which unfortunately did not test any functions from the codebase.

In all honesty, Joseph and Yue have worked together for a long time. They have had many classes together at BU and worked on at least 4 projects together before. The synergy between the two was crucial to be able to produce a working product given the total lack of contributions from Nitin. It worked exceptionally well to be able to let each person handle all aspects of a feature. Integration between features was quite easy as there was excellent communication and trust between Joseph and Yue. However, keeping general responsibilities of frontend vs backend was important. This made it clear who was responsible for managing the service providers on each end, although both Yue and Joseph worked together to ensure seamless integration between all service providers. Overall, it is quite fortunate that Joseph and Yue work so well together to overcome the challenges imposed by Nitin.

Architecture (Updated)

The GradAid architecture is designed to prioritize maintainability, scalability, and flexibility, achieved through a modular and component-based approach. The product is hosted online and uses the classic model–view–controller (MVC) architectural pattern. With modern JavaScript frameworks, the view and controller are extremely well integrated, often coexisting within the same file of code. This has resulted in the common phrases “frontend” and “backend”. Here, the view and controller are referred to as the frontend and the model is referred to as the backend.

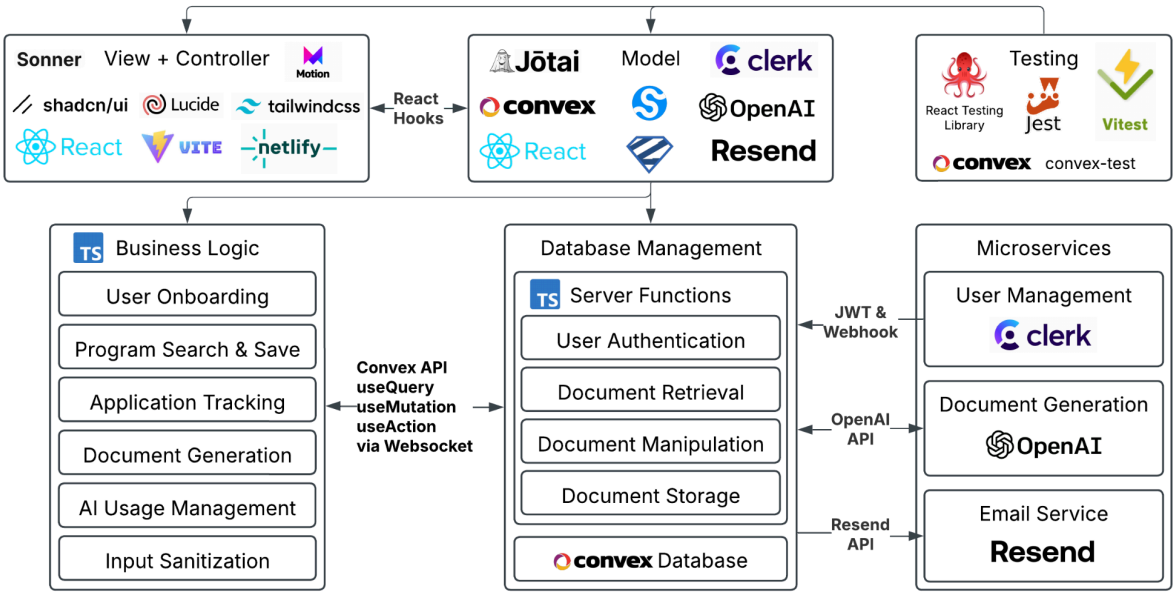
The frontend, built with React, leverages Vite as the build tool and Netlify as the deployment platform for efficient development. Reactive components are build with Shadcn, Framer Motion, Lucide, and Tailwind CSS, ensuring a modern and consistent design. The frontend consists of several modules: a graduate program module, allowing the user to search for programs, save them to their profile, then begin an application to selected programs; the document editor, providing a platform to generate document content with AI or edit existing content; the dashboard, displaying saved documents, application insights, and important user information in a centralized location; onboarding and profile, handling user profile creation and updates;

application management, allowing users to track the progress of their applications and access related documents; and support, providing users with additional information and allowing them to contact the GradAid team. Each of these modules contain individual pages that accomplish one task each, providing users with a simple way to navigate through the website without overwhelming them. Each webpage contains helpful information and links to other related pages to aid users with the workflow of the application. All pages utilize reusable components to ensure a consistent design language and user experience across the website.

The backend, built with Convex, handles data storage and retrieval. Convex uses TypeScript and is built to integrate naturally with React, greatly speeding up development. Convex offers type safety across the codebase by allowing developers to define a schema and write server functions in TypeScript directly in the same repo as the React code. The Convex API provides TypeScript types and three hooks (useQuery, useMutation, and useAction) to reactively interact with the database through server functions. This makes development much simpler, as these hooks dynamically update components as the database changes. The useAction hook is particularly useful as it allows calls to APIs, such as the OpenAI API, then saves the result directly to the database which instantly updates the view. User authentication is handled by Clerk, providing secure user management. When a user first signs up, their information is communicated from Clerk to the Convex database via webhook (Svix), which is stored in the users table. Clerk communicates with Convex during runtime with JSON Web Tokens (JWT), which are used to validate that the user is authenticated and restrict access to personal data. All personal information stored in the database has the unique user ID stored in the record, which is used to verify that the user requesting access to the information has permission (users can only access their own data). This also allows users to delete all personal information from the database. Further security is provided in the model layer by ensuring input validation and prompt sanitization with Zod. Zod ensures that data inputs from users are sensible and free from potential injection attacks such as SQL injection (even though Convex is not a SQL database) and cross-site scripting (XSS). There are additional protections on the backend to prevent malicious access to the database, implementing a multilayered approach that improves security.

GradAid uses three microservices: Clerk, OpenAI, and Resend. As previously discussed, Clerk provides user management and authentication. OpenAI is used to generate customized document content. Although originally GradAid was supposed to use a custom fine-tuned LLM for this task, there is no evidence that Nitin ever trained a model. Endless excuses were provided for why the custom model he supposedly trained couldn't be hosted on a server or integrated with the GradAid codebase. As a result, application documents are generated by using a custom prompt and OpenAI's API to customize them to the user's background, personal goals, and program details. Finally, Resend is used as a microservice to send emails. Resend has an API that provides a simple way to send customized emails, which is used for the contact page and could be used to email letters of recommendation to recommenders in the future. The feature to directly send a letter of recommendation to the recommender was not implemented as there is not a simple way to prevent abuse of the email system.

The GradAid architecture provides several benefits. It simplifies data management and API interactions, leading to more efficient development. All code is written in TypeScript, improving compatibility between components, enhancing error handling, and simplifying debugging. React hooks are used for efficient component updates. The frontend is deployed using Netlify, while the backend and database functionalities are handled by Convex. The modular design and component-based approach continue to promote scalability and flexibility, allowing for easier addition of new features and adaptation to changing requirements.



Security, Privacy, and Reliability (New)

Security and privacy are at the core of GradAid's design. Access to any part of the website other than the landing page requires authentication through Clerk. Security and privacy are ensured by requiring authentication, so the user's identity is always known while using the application. This enables protections to ensure that users may only ever access their own personal information. Several checks are put in place whenever data is queried or mutated to ensure that the user owns the data they are trying to access. The only information stored in the database that is public are the programs and universities. These kinds of records do not contain any personal information, so there is no need for security checks. Additionally, users can delete their account and all associated personal information at any time, providing increased security and privacy.

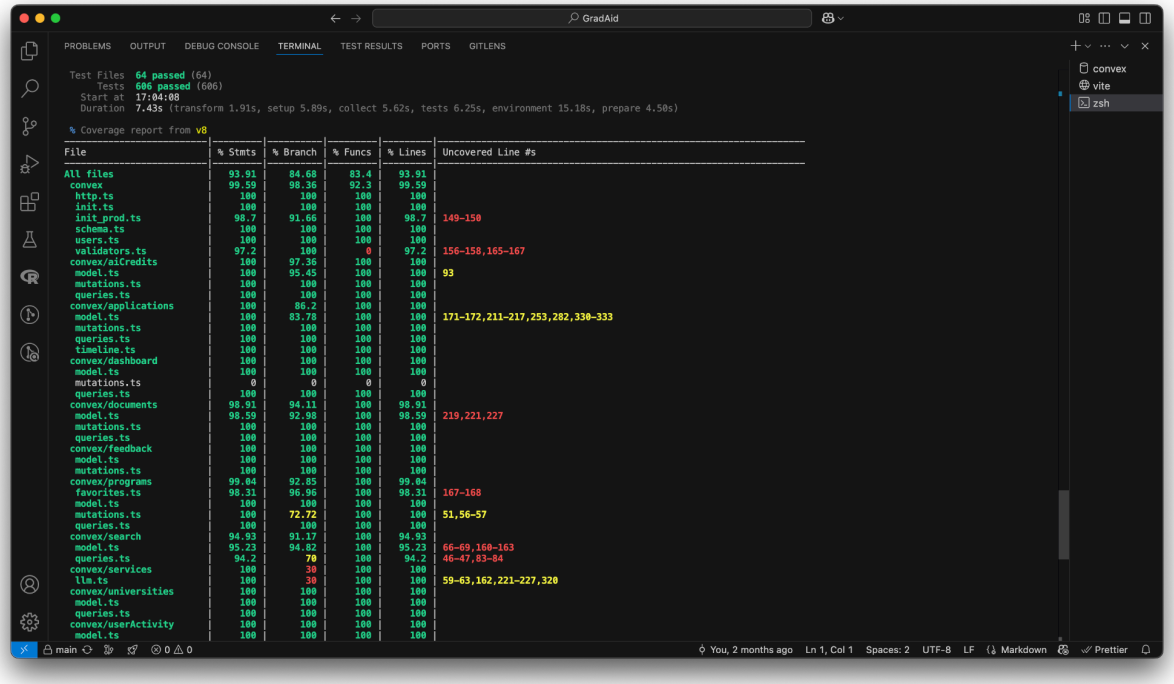
Reliability is ensured by using trusted providers to host all infrastructure. The frontend code is bundled with Vite and hosted by Netlify. Netlify is a trusted platform that ensures GradAid is always available to users via the internet. The backend code is hosted as server functions by

Convex, which also hosts the database. GradAid currently uses the free tier for all services, which puts limits on the bandwidth. Currently the bandwidth is sufficient for the low traffic, but each service can theoretically be scaled without limit if there is massive increase in demand. This allows the website to be resilient and handle high traffic loads. Furthermore, Netlify and Convex both implement rate limiting to prevent malicious attacks such as distributed denial-of-service (DDoS) or cross-site scripting (XSS) from causing significant damage.

Currently there are no strategies to mitigate reliance on certain microservices. If these microservices become unavailable, it will not be simple to replace them. OpenAI is the easiest to swap out with a different LLM API, but Clerk and Resend are fairly well integrated within the codebase. This is a weakness of using microservices that we have not accounted for. However, the microservices offer much greater security and reliability than attempting to implement the functionality directly. Clerk provides top-notch security for user management and Resend handles all of the necessary email security concerns. A future direction for the GradAid architecture would be to reduce reliance on specific microservices and make it easier to swap to a new microservice in the future.

Testing (New)

GradAid has a suite of over 600 unit, UI, and integration tests. Unfortunately, testing was implemented later in development, so there is no end-to-end testing and minimal integration testing. However, code coverage is quite high (83% of functions and 94% of lines of code), ensuring that nearly all critical code has some protection against breaking changes. There is always more work to be done when it comes to writing high quality tests that cover all edge cases, but the existing tests are very comprehensive. GradAid uses Vitest for all testing. Frontend code is tested with the React Testing Library which uses Vitest in the jsdom environment, while backend code is tested with the convex-test library that runs on top of Vitest to create a mock database using the edge-runtime environment. Testing coverage is calculated using the V8 library for Vitest. End-to-end testing was attempted with WebdriverIO, but there was not enough time to get it working properly with authentication. A future direction for GradAid testing is to fully implement end-to-end testing with WebdriverIO or a similar package.



Team Profile (Unchanged)

Yue Zhang:

Yue is developing the frontend, ensuring a smooth and intuitive user experience. He also works on backend API integration and user authentication, enabling secure sign-in and seamless communication between the UI and database. He ensures application security by implementing input validation on all website components.

Nitin Krishna Bojji:

Works with LLM and agentic AI to integrate these technologies into our platform, automating the creation of SOPs and LORs for master's students. His focus is on enhancing performance, scalability, and security to ensure a responsive and reliable user experience.

Joseph Rissman:

Joseph's focus for this project is developing and maintaining the database, integrating it with the other components to ensure a smooth flow of data. He is working to ensure user data security by implementing role access control measures at the database level.

Product Vision (Unchanged)

For international students planning to attend graduate school in the US who need to draft documentation required for their applications. GradAid is an AI Driven Service Provider that provides students with freedom to be creative and tailor stories to their individual needs at a low-cost. Unlike educational consultancies, our product is available to users at any time and at a lower cost.

Personas (Unchanged)

Persona 1 - Li Lei

Li Lei, age 24, is a Chinese graduate student completing his MS degree in Biomedical Science. Originally from China, he earned his bachelor's degree in Chemistry before pursuing his master's studies. Li is passionate about scientific research and is determined to pursue a PhD in the United States to further his academic career. Despite his strong research background, he struggles with academic writing in English and is unfamiliar with the specific expectations of U.S. graduate applications. Li hopes to craft compelling Statements of Purpose (SOPs) and Letters of Recommendation (LORs) that effectively showcase his research achievements and align with his PhD aspirations. He views GradAid as an essential tool to generate professional, grammatically accurate documents tailored to his needs while helping him refine his tone and clarity for a natural-sounding application.

Persona 2 - Astha Jain

Astha Jain, a 28-year-old from Jaipur, India, comes from a large family and was the first to attend college. She pursued her passion for medicine at Mahatma Gandhi University of Medical Sciences and Technology, earning a Bachelor of Medicine and Bachelor of Surgery (MBBS) degree with high marks. Although she initially aspired to become a surgeon, she faced challenges finding employment immediately after graduation. Relocating to New Delhi in search of opportunities, she worked as a server to support herself while continuing her job search. Eventually, she secured a position as a junior doctor at a family medicine practice, where she worked for two years until the practice closed due to financial difficulties. Following this setback, she accepted a nursing position at Jeewan Mala Hospital in New Delhi. This career detour, coupled with the demanding hours of her new role, has prompted her to re-evaluate her career path. She now plans to pursue a graduate degree in a growing and more lucrative field, such as computer science or finance. Having saved enough money to study in the United States, she hopes that a graduate degree from a top American university will open doors to high-paying jobs, enabling her to support her aging parents, whose ability to work is diminishing.

Unsure of which specific field to pursue, Astha is finding it difficult to identify suitable programs. Even when she finds a seemingly appropriate program, she struggles to articulate her diverse experiences in a compelling Statement of Purpose (SOP). While acknowledging that her career journey has been unconventional, she recognizes the valuable lessons and transferable skills she has acquired along the way. She believes that an educational consultant could be

instrumental in navigating the application process by understanding her unique story, her personal journey, and her future ambitions.

Persona 3 - Carlos Mendoza

Carlos Mendoza, age 26, is a software engineer from Mexico City with four years of experience in backend development. He earned his bachelor's degree in Computer Science from UNAM and has worked at a mid-sized tech company. Carlos has always been passionate about AI and machine learning and wants to pivot into research, but his current job focuses more on software development rather than deep AI work.

He plans to apply for an MS in Artificial Intelligence in the United States to gain the necessary academic background to transition into AI research roles. However, he is unfamiliar with the research-heavy SOPs required for graduate applications and has limited experience writing formal academic documents in English. He is also unsure how to best frame his work experience to demonstrate its relevance to AI research. He hopes GradAid will help him craft a strong SOP that highlights his technical background while showcasing his research potential in AI.

Scenarios (Unchanged)

Scenario 1 - Li Lei

Li Lei, a 24-year-old Chinese graduate student completing his MS in Biomedical Science, is preparing to apply for PhD programs in the United States. With a BS in Chemistry and extensive research experience, he is eager to continue his academic journey in biomedical innovation. However, he faces significant challenges in crafting a compelling application due to his limited English proficiency and unfamiliarity with the tone, structure, and expectations of U.S. graduate admissions. Writing a strong Statement of Purpose (SOP) and securing well-crafted Letters of Recommendation (LORs) feels overwhelming, as he struggles to articulate his research achievements and career goals in a way that resonates with admissions committees.

To overcome these challenges, Li turns to GradAid, which helps him generate well-structured, grammatically accurate SOPs and LORs tailored to his background and aspirations. By inputting key details about his academic journey and research experience, he receives personalized drafts that he can refine using GradAid’s tone adjustment and clarity-enhancing features. The platform also provides research-focused suggestions to ensure his documents align with the expectations of specific PhD programs. With GradAid’s support, Li gains confidence in his application, knowing that his materials effectively highlight his strengths and are presented in a professional, natural-sounding manner.

Scenario 2 - Astha Jain

Astha is determined to pursue a graduate degree in the US, but the application process feels overwhelming. Remembering a friend's recommendation, she opens her browser and navigates

to GradAid.com. Astha clicks the "Get Started" button and begins interacting with GradAid's AI chatbot.

GradAid asks her a series of questions: her background, academic achievements, work experience, career aspirations, and preferred fields of study. Astha appreciates how GradAid's conversational interface makes it easy to share her story, even the less conventional parts. She explains her medical background, her shift to nursing, and her desire to transition into a more lucrative and stable career to support her parents. Based on Astha's input, GradAid's AI analyzes her profile and generates a list of potential graduate programs in computer science and finance at universities across the US. The recommendations are tailored to her academic record, work history, and career goals. Astha is impressed by the breadth and relevance of the suggestions.

Astha selects a few programs that pique her interest. She knows the Statement of Purpose (SOP) is crucial, and it's where she's been struggling the most. Thankfully, GradAid offers an SOP brainstorming tool. GradAid's AI generates a series of prompts and potential themes for her SOP, suggesting ways to connect her seemingly disparate experiences into a cohesive narrative. Astha finds this incredibly helpful, as she's able to recognize the transferable skills she's gained, like problem-solving, communication, and adaptability.

Scenario 3 -Carlos Mendoza

Carlos has started drafting his Statement of Purpose (SOP) for MS programs in AI, but he's struggling to connect his industry experience in backend development to his passion for machine learning. Unsure of how to frame his background, he logs into GradAid and inputs details about his education, work experience, and research interests.

GradAid analyzes his profile and suggests ways to highlight transferable skills from his software engineering background, such as his experience working with large datasets, optimization algorithms, and distributed computing. The AI-generated SOP draft weaves his technical experience into a compelling narrative, emphasizing how his work has naturally led him toward AI research. The platform also offers AI-powered editing tools to refine the language, ensuring his SOP is both professional and persuasive.

Scenario 4 - Carlos Mendoza

Carlos has shortlisted five MS programs in AI, each with slightly different focuses—some emphasize research, while others are more industry-oriented. He wants to tailor his SOP for each program but doesn't know how to efficiently modify his content.

Using GradAid's customization feature, he selects the target programs and receives program-specific suggestions for refining his SOP. The tool highlights key faculty members and research labs he should mention, recommends how to emphasize certain skills based on each program's focus, and adjusts the wording to align with the expectations of research vs. industry-focused programs. With GradAid's help, Carlos creates tailored versions of his SOP, increasing his chances of securing admission.

Technical Risks (Updated)

The Admissions Alchemists team has successfully navigated the challenges inherent in developing a web-centric application with limited prior web development experience. The team has made significant strides in acquiring new skills, particularly in React, Convex, and UI development with Shadcn and Tailwind CSS.

The team effectively utilized collaborative learning strategies such as pair programming and knowledge-sharing sessions to accelerate this process. For example, Joseph provided guidance on React hooks, Convex queries, and TypeScript best practices, while the team worked together to resolve environment setup and data handling issues. The team has mitigated the risk by maintaining its commitment to proactive learning, leveraging online resources and seeking guidance from experienced mentors as needed.

The team's decision to adopt React, Convex, and related technologies presented an initial risk due to the team's unfamiliarity. However, the team ultimately overcame this risk, becoming proficient in TypeScript and learning the React/Convex ecosystem. The team's collaborative approach, characterized by regular progress reviews and focused meetings, has been instrumental in overcoming this challenge.

Time constraints, stemming from team members' diverse commitments, remained a persistent challenge. The team's strategy of dividing the project into short, focused iterations with clearly defined milestones has proven effective in managing this risk. By prioritizing key functionalities and adhering to deadlines, the team has maintained a steady pace of progress. The agile mindset adopted by the team, emphasizing flexibility and adaptability, has also been crucial in responding to evolving requirements and unforeseen challenges. For example, the team adjusted its development plan to accommodate the migration to Convex and address critical bugs. Regular workload assessments and adjustments were necessary to ensure timely delivery.

Adding to the complexity, several team members have faced health challenges during this semester, impacting their ability to contribute consistently. The team is adapting by providing support and adjusting workloads to accommodate these unforeseen circumstances. Open communication and flexible task management are crucial to minimizing the impact of these health-related disruptions on the project's progress.

Effective communication and collaboration have been paramount in mitigating these risks. The established centralized communication channel and regular meetings have facilitated seamless information exchange and coordination. The team's culture of open communication and mutual support has fostered a collaborative environment, empowering team members to contribute effectively.

Overall, the Admissions Alchemists team has demonstrated a strong capacity to adapt and overcome technical challenges. The team's commitment to continuous learning, collaboration, and agile methodologies has been instrumental in navigating the project's complexities.

Ultimately the team was able to deliver the MVP, even without any contributions from one of the team members. This is a testament to the resilience and adaptability of the team.

Product Demonstration (Updated)

GradAid is available on the internet, designed for both desktop and mobile.
https://gradaid.online/